

ASSIGNMENT 10.1

Name: Kasuri Rohith

Batch: 23

Hall no: 2303a510i2

Task Description #1 – Syntax and Logic Errors

Task: Use AI to identify and fix syntax and logic errors in a faulty Python script.

Sample Input Code:

```
# Calculate average score of a student

def calc_average(marks):

total = 0

for m in marks:

total += m

average = total / len(marks)

return avrage # Typo here

marks = [85, 90, 78, 92]

print("Average Score is ", calc_average(marks))
```

Expected Output:

- Corrected and runnable Python code with explanations of the fixes.

Code:

```
demo1.py > ...
1 # refactored code with a typo and a missing parenthesis
2
3 def calc_average(marks):
4     total = 0
5     for m in marks:
6         total += m
7     average = total / len(marks)
8     return average # Fixed typo : 'avrage' to 'average' and added missing parant
9
10 marks = [85, 90, 78, 92]
11 print("Average Score is ", calc_average(marks))
```

Output:

```
11.exe c:/Users/rohit/OneDrive/Desktop/py1/demo1.py
Average Score is 86.25
PS C:\Users\rohit\OneDrive\Desktop\py1>
```

Task Description #2 – PEP 8 Compliance

Task: Use AI to refactor Python code to follow PEP 8 style guidelines.

Sample Input Code:

```
def area_of_rect(L,B) : return L*B

print(area_of_rect(10,20))
```

Expected Output:

- Well-formatted PEP 8-compliant Python code.

Code:

```

1 C:\Users\rohit\OneDrive\Desktop\py1\demo1.py *B
2 print(area_of_rect(10,20))
3
4 #refactored the above code and add documentation and type hints
5 def area_of_rect(length: float, breadth: float) -> float:
6     """
7     Calculate the area of a rectangle given its length and breadth.
8
9     Parameters:
10    length (float): The length of the rectangle.
11    breadth (float): The breadth of the rectangle.
12
13    Returns:
14    float: The area of the rectangle calculated as length multiplied by breadth.
15
16    Raises:
17    ValueError: If length or breadth is negative, as dimensions cannot be negative.
18    TypeError: If length or breadth is not a number (int or float).
19    """
20    if not isinstance(length, (int, float)) or not isinstance(breadth, (int, float)):
21        raise TypeError("Length and breadth must be numbers (int or float).")
22    if length < 0 or breadth < 0:
23        raise ValueError("Length and breadth must be non-negative.")
24    return length * breadth
25 print([area_of_rect(10, 20)])

```

Output:

```

PS C:\Users\rohit\OneDrive\Desktop\py1> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.
11.exe c:/Users/rohit/OneDrive/Desktop/py1/demo1.py
200
200
PS C:\Users\rohit\OneDrive\Desktop\py1>

```

Task Description #3 – Readability Enhancement

Task: Use AI to make code more readable without changing its logic.

Sample Input Code:

```

def c(x,y):
    return x*y/100

a=200
b=15

print(c(a,b))

```

Expected Output:

- Python code with descriptive variable names, inline

comments, and clear formatting

code:

```
1  def c(x,y):
2      return x*y/100
3
4  a=200
5  b=15
6  print(c(a,b))
7
8  #refactored the above code with descriptive variable names, inline comments, and
9  def calculate_percentage(part: float, whole: float) -> float:
10     """
11     Calculate the percentage of a part relative to a whole.
12
13     Parameters:
14     part (float): The portion or part value.
15     whole (float): The total or whole value.
16
17     Returns:
18     float: The percentage calculated as (part / whole) * 100.
19
20     Raises:
21     ValueError: If the whole is zero, as division by zero is not allowed.
22     TypeError: If part or whole is not a number (int or float).
23     """
24     if not isinstance(part, (int, float)) or not isinstance(whole, (int, float)):
25         raise TypeError("Both part and whole must be numbers (int or float).")
26     if whole == 0:
27         raise ValueError("Whole cannot be zero to avoid division by zero.")
28     return (part / whole) * 100
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\py1> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/py1/demo1.py
30.0
PS C:\Users\rohit\OneDrive\Desktop\py1>
```

Task Description #4 – Refactoring for Maintainability

Task: Use AI to break repetitive or long code into reusable functions.

Sample Input Code:

```
students = ["Alice", "Bob", "Charlie"]
print("Welcome", students[0])
```

```
print("Welcome", students[1])
```

```
print("Welcome", students[2])
```

Expected Output:

- Modular code with reusable functions.

Code:

```
demo1.py > welcome_student
1  students = ["Alice", "Bob", "Charlie"]
2  print("Welcome", students[0])
3  print("Welcome", students[1])
4  print("Welcome", students[2])
5
6  #refactored code to reduce redundancy with reusable function
7  def welcome_student(student: str) -> None:
8      """
9      Print a welcome message for a student.
10
11      Parameters:
12      student (str): The name of the student to welcome.
13
14      Returns:
15      None
16      values:
17      student: A string representing the name of the student.
18      type error: If the input is not a string, a TypeError will be raised.
19      """
20      if not isinstance(student, str):
21          raise TypeError("Student name must be a string.")
22      print("Welcome", student)
```

Output:

```
PS C:\Users\rohit\OneDrive\Desktop\py1> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/py1/demo1.py
30.0
PS C:\Users\rohit\OneDrive\Desktop\py1>
```

Task Description #5 – Performance Optimization

Task: Use AI to make the code run faster.

Sample Input Code:

Find squares of numbers

```
nums = [i for i in range(1,1000000)]
```

```
squares = []
```

```
for n in nums:
```

```
    squares.append(n**2)
```

```
print(len(squares))
```

Expected Output:

- Optimized code using list comprehensions or vectorized operations.

Code:

```
demo1.py > ...
1  nums = [i for i in range(1,1000000)]
2  squares = []
3  for n in nums:
4      squares.append(n**2)
5  print(len(squares))
6
7  #refactored the above code to reduce time complexity
8  nums = [i for i in range(1,1000000)]
9  squares = [n**2 for n in nums]
10 print(len(squares))
```

Output

```
PS C:\Users\rohit\OneDrive\Desktop\py1> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/py1/demo1.py
999999
999999
PS C:\Users\rohit\OneDrive\Desktop\py1>
```

Task Description #6 – Complexity Reduction

Task: Use AI to simplify overly complex logic.

Sample Input Code:

```
def grade(score):
```

```
if score >= 90:
```

```
    return "A"
```

```
else:
```

```
    if score >= 80:
```

```
        return "B"
```

```
    else:
```

```
        if score >= 70:
```

```
            return "C"
```

```
        else:
```

```
            if score >= 60:
```

```
                return "D"
```

```
            else:
```

```
                return "F"
```

Expected Output:

- Cleaner logic using elif or dictionary mapping.

Code:

```

1  def grade(score):
2      if score >= 90:
3          return "A"
4      else:
5          if score >= 80:
6              return "B"
7          else:
8              if score >= 70:
9                  return "C"
10             else:
11                 if score >= 60:
12                     return "D"
13                 else:
14                     return "F"
15
16 #refactored code to cleaner logic using elif or dictionary mapping.
17 def grade(score: int) -> str:
18     """
19     Return the grade based on the score.
20     Parameters:
21     score (int): Student score
22     Returns:
23     str: Grade (A, B, C, D, or F)
24     """
25     if score >= 90:
26         return "A"
27     elif score >= 80:
28         return "B"
29     elif score >= 70:
30         return "C"
31     elif score >= 60:
32         return "D"
33     else:
34         return "F"
35
36 print(grade(95))
37
38 def grade(score: int) -> str:
39     """
40     Return the grade based on the score using dictionary mapping.
41     """
42     grade_map = {
43         90: "A",
44         80: "B",
45         70: "C",
46         60: "D",
47         0: "F"
48     }
49
50     for cutoff, letter in grade_map.items():
51         if score >= cutoff:
52             return letter
53
54 print(grade(85))

```

Output:

```

● PS C:\Users\rohit\OneDrive\Desktop\py1> & C:/Users/rohit/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/rohit/OneDrive/Desktop/py1/demo1.py
A
B
○ PS C:\Users\rohit\OneDrive\Desktop\py1>

```